A Computer Vision Project Report

MIDTERM PROJECT REPORT

Submitted as a requirement for the course
Computer Vision

Group Project Specialization

Object Sorting

Prepared by:

Alba, John Raymond S.
Bacsain, Jave A.
Bagaporo, John Louie B.
Nonato, Francis Gabriel F.
Palacios, Harvey Lloyd V.
Prestado, Marc Justin N.

BS Computer Science – 4A

Midterm_Object_Sorting_RealWaste.ipynb

Submitted to:

Ms. Elena Clarexe R. Gulane
Course Instructor

1. INTRODUCTION

1.1 Project Overview

This project develops the computer vision component of a vision-based autonomous 4WD robot assigned to the Object Sorting specialization. Currently, the proposed system asks one question: can the system itself see and recognize objects in front of the camera? Given an image or a live camera frame of a waste item, the system must identify which type of material it is and show that result visibly on screen. The robot is not required to move at this stage. In the Final stage, the recognized class will drive the robot's decision and motor control so that it can physically sort objects.

The Computer Vision problem addressed in this project is a multi-class image classification. We train a convolutional neural network (CNN) on the RealWaste dataset (Single et al., 2023), sourced from UC Irvine Machine Repository, a public collection of 4,752 photographs of waste items taken at the point of reception at a landfill in Wollongong, Australia. Unlike studio-quality datasets, these images show objects in cluttered, uneven, real-life conditions, which makes them a more realistic test of whether a classifier can handle the visual variation a sorting robot would face.

1.2 Problem Description

Sorting waste by material is a repetitive task that is slow, inconsistent when done by hand, and unpleasant or unsafe for the people doing it. Mistakes contaminate recyclable material and reduce its value. An automated sorter first needs a reliable way to tell what an object is from a camera image alone.

This is difficult for a computer vision system for several reasons. Items of the same class vary widely in shape, size, color and condition (a crushed bottle and an intact bottle are both plastic). Different classes can look alike (paper and cardboard share color and texture; clear glass and clear plastic reflect light similarly). Items may also be partly hidden, deformed, dirty, or photographed under changing lighting and backgrounds.

The current problem statement for this project would be: given a single RGB image of one waste item, the system must assign it to exactly one of the selected classes, report a confidence score, and generalize to images it has not seen during training.

1.3 Project Objectives

General objective:

To design, train and evaluate a computer vision system that recognizes selected waste object classes from images and live camera input, forming the vision foundation of an object-sorting robot.

Specific objectives:

- To prepare a dataset of five waste classes with at least 100 images per class, divided into separate training, validation and testing sets.
- To apply image preprocessing (resizing, color conversion, normalization) and data augmentation to prepare images for the model.
- To train a CNN classifier using transfer learning and tune it using the validation set.
- To evaluate the model on the held-out test set using accuracy, precision, recall, F1-score and a confusion matrix, with a target of at least 90% overall test accuracy.
- To demonstrate the complete pipeline (Camera/Image → Preprocessing → Model → Recognition → Visual Output) on images chosen by the instructor and on live camera input, displaying the predicted class label and confidence score.
- To check generalization on additional images captured by the group under different lighting, backgrounds and viewing angles that were not used in training.

1.4 Scope and Limitations

The current project progress covers the Computer Vision stage of Object Sorting for the Midterm. It does not cover robot motion or sorting mechanics.

- **Classes/categories.** Five material classes from the RealWaste dataset: Cardboard, Glass, Metal, Paper and Plastic. The dataset contains nine classes; the other four (Food Organics, Miscellaneous Trash, Textile Trash and Vegetation) are excluded to remain within the Midterm limit of three to five classes. The subset contains 3,092 images, well above the minimum of 100 per class.

Class	Total images	Training (70%)	Validation (15%)	Testing (15%)
Cardboard	461	323	69	69
Glass	420	294	63	63
Metal	790	553	118	119
Paper	500	350	75	75
Plastic	921	644	139	138
Total	**3,092**	**2,164**	**464**	**464**

Table 1. Class distribution and planned 70/15/15 stratified split of the RealWaste subset.

- **Objects/targets**. A single waste item per image, belonging to one of the five classes above, presented reasonably close to the camera.
- **Visual tasks.** Image classification: predicting the class label and a confidence score for the whole image.
- **Operating scope**. Still images from the dataset, instructor-provided images, and live camera input of items held or placed in front of the camera in an indoor setting with reasonable lighting. Processing runs on a laptop/PC.

Relevant limitations.

- The training images come from one landfill facility and one camera setup, so accuracy may drop on images with very different backgrounds, lighting or camera quality.
- The system can only output one of the five trained classes. An object outside these classes (for example food waste or fabric) will still be forced into the closest of the five.
- The dataset is imbalanced (Plastic has 921 images and Glass 420), which may bias predictions toward larger classes unless handled during training.
- Visually similar classes (Paper and Cardboard, Glass and Plastic) are expected to be the main source of errors.

1.5 Group Project Specialization

The group's assigned specialization is **Object Sorting**. The required Computer Vision task is to recognize the selected object classes so that a robot can later sort them. The system must correctly identify the selected classes from images and live camera input and show a visible result (class label and confidence score). The recognition result will be used for decision logic and motor control to move and sort objects autonomously.

2. SYSTEM DESIGN AND ARCHITECTURE

2.1 Overall System Architecture

Figure 1 shows the system architecture of our project. Part A is the pipeline run at demonstration time: an input image or camera frame is preprocessed, passed to the trained CNN, converted to a class prediction with a confidence score, and displayed on the frame. Part B is the offline pipeline that produces the trained model: the dataset subset is split, preprocessed and augmented, used to train the network, and finally evaluated on the test set.

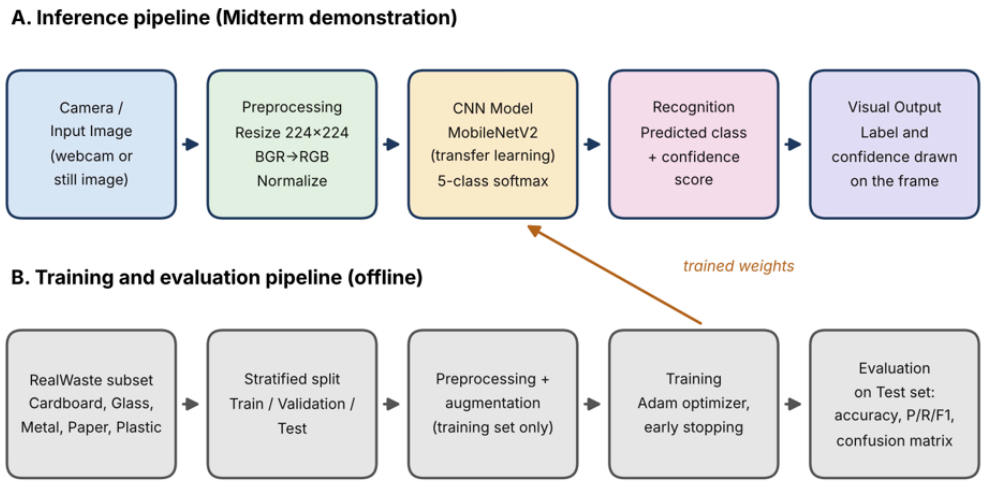


Figure 1. System Architecture for Object Sorting

2.2 Computer Vision Workflow

The Computer Vision system performs the following sequence of operations:

1. **Dataset preparation.** Select the five classes from RealWaste (524 × 524 pixel color images) and verify that images are readable and correctly labeled.
2. **Dataset splitting.** Divide each class into training, validation and testing sets using a stratified 70/15/15 split. The split is done before augmentation so that no test or validation image (or a modified copy) is seen during training.
3. **Preprocessing.** Resize every image to 224 × 224 pixels, ensure RGB channel order, and scale pixel values to the range expected by the network.
4. **Data augmentation (training set only).** Apply random flips, rotations, zoom, brightness and contrast changes so the model learns to handle different lighting and viewing angles.
5. **Model construction.** Use MobileNetV2 pre-trained on ImageNet as a feature extractor, followed by global average pooling, dropout, and a dense softmax layer with five outputs.
6. **Training.** Train the new classification layers with the Adam optimizer and categorical cross-entropy loss, apply class weights to reduce the effect of class imbalance, then fine-tune upper layers at a lower learning rate. Training stops early when validation loss stops improving.
7. **Evaluation.** Run the final model once on the untouched test set and compute accuracy, per-class precision, recall and F1-score, and a confusion matrix. Additional group-captured images with different lighting and backgrounds are used as unseen test data.
8. **Recognition and visual output.** For a new image or live camera frame, apply the same preprocessing, run the model, take the class with the highest softmax probability, and draw the class label and confidence score on the image. Predictions below a confidence threshold are marked as uncertain.

2.3 Hardware Components

Listed below are the proposed hardware components relevant to the Computer Vision prototype.

Component	Specification / Model	Purpose
Camera	USB webcam, 720p (or laptop built-in camera)	Captures live frames of the object to be recognized
Controller	Laptop/PC (Midterm); Arduino/ESP32 planned for Final	Runs preprocessing and model inference; later sends motor commands
Robot Chassis	4WD chassis kit (Final stage)	Mobile platform for sorting; not needed for Midterm demo
Motor Driver	L298N dual H-bridge (Final stage)	Drives the DC motors from controller signals

Motors	4 × DC gear motors (Final stage)	Moves the robot
Power Source	Laptop power / battery pack	Powers the processing unit and, in Final, the motors
Others	Sample waste items (bottle, can, paper, cardboard, glass); plain background; adequate lighting	Live demonstration and capture of unseen test images

2.4 Software and Development Tools

The system is implemented in Python using open-source libraries. All external datasets, libraries and pre-trained models are acknowledged in the References.

Software / Tool	**Version**	**Purpose**
Python	3.11.9	Main programming language for data preparation, training, evaluation and the demonstration program
Arduino IDE		Reserved for Final motor-control programming
TensorFlow	2.21.0	Deep learning framework. Loads and resizes images (tf.data, tf.image), trains the CNN and runs predictions
Keras (part of TensorFlow)	3.15.1	High-level API used to build the model (augmentation layers, pooling, dropout, softmax), compile it, train it with EarlyStopping/ReduceLROnPlateau, and save it as .keras
MobileNetV2 (pre-trained on ImageNet)	Keras Applications, 224×224, include_top=False	Pre-trained CNN backbone used for transfer learning and fine-tuning
OpenCV (opencv-python)	5.0.0 (package 5.0.0.93)	Reads images, converts BGR→RGB, resizes for the demo, and draws the label, confidence and box on the output image. Also supports webcam frames via cv2.VideoCapture
NumPy	2.4.6	Array math: labels, probabilities, argmax, normalizing the confusion matrix
pandas	2.2.3	Tables for class counts, the train/val/test split and the classification report. Exports split.csv

Matplotlib	3.11.1	Plots the class distribution, sample images, augmentation preview, training curves, confusion matrix and prediction results
scikit-learn	1.9.1	Stratified train/val/test split (train_test_split), balanced class weights, and evaluation (accuracy, classification report, confusion matrix)
RealWaste dataset	UCI ML Repository, 2023 (DOI 10.24432/C5SS4G, CC BY 4.0)	Image dataset; 5 classes are used (Cardboard, Glass, Metal, Paper, Plastic), 3,092 images
Python standard library (os, pathlib, zipfile, shutil, json, random, time, glob)	Comes with Python 3.11.9	File handling, extracting the dataset zip, random seeds, timing, saving results.json and class_names.json

REFERENCES

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems* [Computer software]. https://www.tensorflow.org/

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.

Chollet, F. (2015). *Keras* [Computer software]. https://keras.io

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248–255). IEEE. https://doi.org/10.1109/CVPR.2009.5206848

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn:

Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Python Software Foundation. (n.d.). *Python* [Computer software]. https://www.python.org/

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). IEEE. https://doi.org/10.1109/CVPR.2018.00474

Single, S., Iranmanesh, S., & Raad, R. (2023). *RealWaste* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5SS4G

Single, S., Iranmanesh, S., & Raad, R. (2023). RealWaste: A novel real-life data set for landfill waste classification using deep learning. *Information, 14*(12), Article 633. https://doi.org/10.3390/info14120633